

LedgerLens

# Supplier Invoice Reconciliation System Design

|                    |                        |                                    |
|--------------------|------------------------|------------------------------------|
| STATUS<br>Proposed | OWNER<br>Project owner | LAST UPDATED<br>September 18, 2026 |
|--------------------|------------------------|------------------------------------|

|              |                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------|
| Authors      | Project owner                                                                                                         |
| Reviewers    | Technical reviewers                                                                                                   |
| Related docs | Repository README and OpenAPI documentation                                                                           |
| Scope        | Invoice-first document intake, evidence-backed extraction, deterministic reconciliation, and human decision workflow. |

# 1. Abstract

LedgerLens is a procure-to-pay exception-resolution service. It accepts an invoice, stores the original file once by SHA-256 fingerprint, extracts evidence-backed fields, links the invoice to purchasing records, and runs deterministic checks against the purchase order, delivery receipt, and contract. The design makes the system useful for review without allowing extraction output to silently approve a payment.

The local project supports PDF, PNG, JPEG, and TIFF intake; embedded PDF text; optional Tesseract OCR; a rules baseline; optional Groq comparison; and an opt-in fine-tuned local Qwen adapter. It is a learning and portfolio system, not a production payment service: authentication, migrations, tenant isolation, and production retention policy remain future work.

# 2. Goals and Non-Goals

| Goals                                                                                    | Non-goals                                                      |
|------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Keep source, page text, extracted values, and evidence together for audit.               | Automatically release or pay an invoice.                       |
| Detect quantity, price, arithmetic, supplier, and currency exceptions deterministically. | Interpret arbitrary legal contract language.                   |
| Make duplicate uploads and retried jobs safe to run repeatedly.                          | Replace an ERP, document system, or AP workflow.               |
| Require a human to link records and make the final decision.                             | Claim production quality from the current synthetic benchmark. |

# 3. Background and Problem Statement

Accounts-payable review requires comparing four sources: the supplier invoice, purchase order, delivery receipt, and contract. A simple PDF question-answering application can retrieve text, but it does not preserve a structured evidence trail, distinguish extraction failure from business-rule failure, or ensure that a payment decision receives human review. LedgerLens models these records separately so each comparison can name the source of an exception.

The system boundary begins when a reviewer uploads an invoice. The API validates file signatures and persists a document identity; a worker extracts text and schema-validated fields; matching uses saved supplier and PO records; the reconciliation engine compares ordered, accepted, invoiced, and contracted values; and the reviewer decides the case. The guiding invariant is that automated components may propose and explain, but only a reviewer may create purchasing records or approve an exception.

## 4. Proposed Architecture

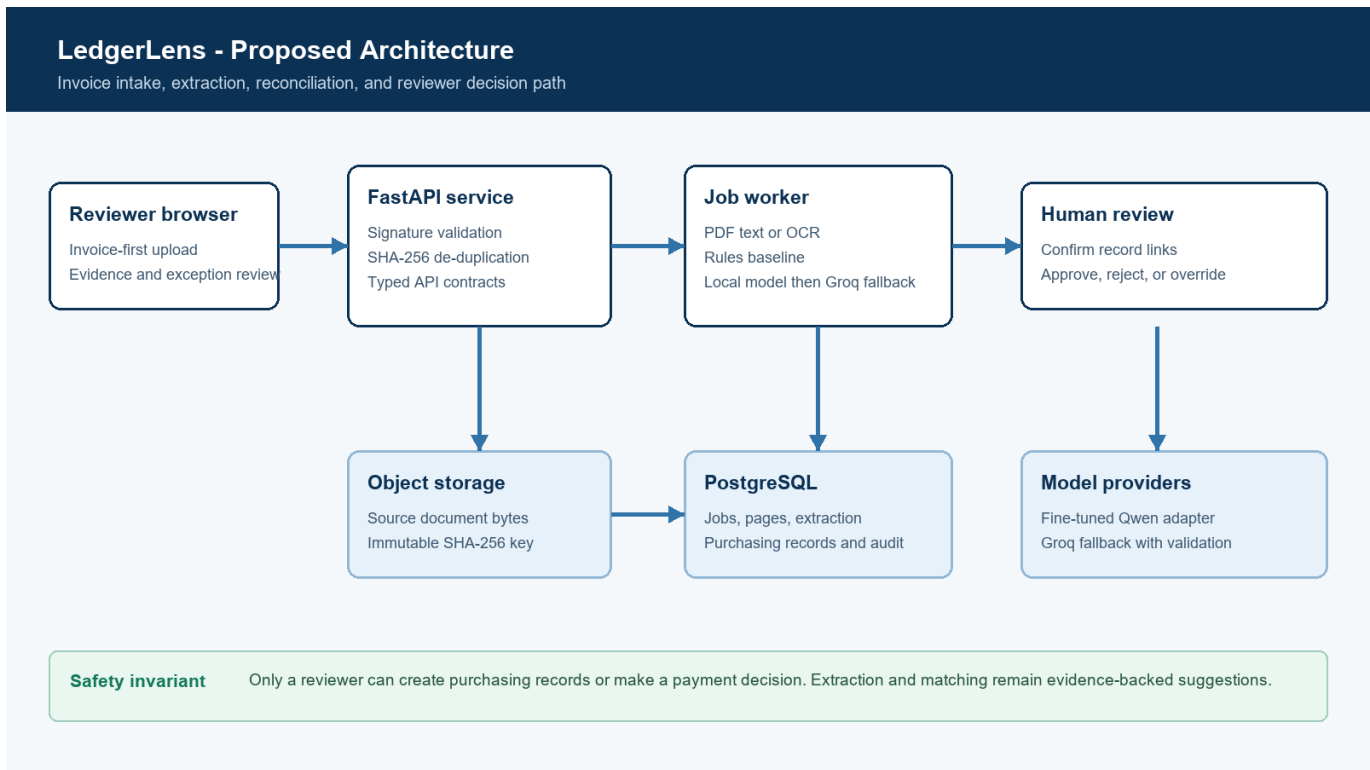


Figure 1. LedgerLens proposed architecture.

| Component             | Responsibility                                                                                | Primary storage                 | Failure behavior                                                                       |
|-----------------------|-----------------------------------------------------------------------------------------------|---------------------------------|----------------------------------------------------------------------------------------|
| Review UI             | Starts with an invoice and requests supporting documents only when saved records are missing. | Browser session                 | Shows job status and does not claim reconciliation is complete before the case exists. |
| FastAPI API           | Validates uploads, exposes typed endpoints, proposes matches, and enforces reviewer gates.    | PostgreSQL and document storage | Rejects invalid inputs; returns explicit missing-record or validation errors.          |
| Extraction worker     | Reads PDF text or OCR and stores pages plus structured output.                                | Jobs and pages                  | Marks job failed with a bounded error; retry is explicit.                              |
| Reconciliation engine | Compares invoice lines with PO, receipts, and active contract tolerance.                      | Purchasing and case tables      | Produces needs_review exceptions instead of changing payment state.                    |

## 5. Request Lifecycle

1. A reviewer uploads an invoice. The API checks the file signature, allowed type, declared document type, and content hash before storing the source bytes and upload attempt.
2. The API creates or reuses a durable queued processing job. The returned job ID lets the UI poll rather than assuming extraction finishes in the request.
3. The worker claims one queued job with row locking. It reads embedded PDF text when available and uses OCR for scanned documents when Tesseract is installed.
4. The extractor produces typed invoice fields only when supporting page text is available. The rules result is the stored baseline; Groq and the local adapter are comparison or configured-backend paths validated against the same evidence rule.

5. The API proposes exact supplier and PO matches, checks duplicates, and reports which supporting record is missing. The UI asks for the next document only when that record cannot be resolved from saved data.
6. A reviewer confirms links or saves a purchasing record. The reconciliation engine then checks supplier, currency, PO line, ordered quantity, accepted quantity, unit price tolerance, and line arithmetic.
7. The service stores the report and audit event. A clean report can be approved; a case with blocking exceptions requires a documented override or rejection.

## 6. API and Data Contracts

| Field             | Type    | Required             | Description                                                                                         |
|-------------------|---------|----------------------|-----------------------------------------------------------------------------------------------------|
| document_id       | UUID    | Yes                  | Stable identity for stored bytes; ties uploads, jobs, pages, and extraction together.               |
| sha256            | string  | Yes                  | Content fingerprint. Identical bytes reuse one stored document while retaining each upload attempt. |
| document_type     | enum    | Yes                  | invoice, purchase_order, delivery_receipt, or contract.                                             |
| output_json       | object  | No                   | Schema-validated fields, line items, confidence, and page-text evidence.                            |
| extractor_version | string  | Yes after extraction | Identifies rules, Groq, or local model path used.                                                   |
| case_id           | integer | No                   | Created after reviewer confirmation; anchors reconciliation and decisions.                          |
| audit event       | object  | Yes for decisions    | Captures confirmation, reconciliation, approval, rejection, or override activity.                   |

## 7. Consistency, Idempotency, and Replay

A document job becomes complete only after its page text, extraction record, and job metadata are stored in one database transaction. A job with unavailable OCR or invalid structured output is recorded as failed rather than treated as an empty successful extraction.

Document identity is content-addressed by SHA-256. Jobs are claimed from the queued state using database locking, and duplicate invoice numbers are checked within the matched supplier before a reviewer can create an invoice record. The system retains identifiers, versions, evidence, reports, and decision events for investigation and replay.

## 8. Security and Privacy Considerations

The local learning app has no authentication and must not be exposed publicly. A production deployment needs authenticated users, role-based authorization, and tenant isolation in storage and queries.

Source files and extracted page text can contain financial and supplier data. Logs and traces must avoid source text, secrets, and raw document contents. Groq credentials belong in an ignored .env file or managed secret store and must never be committed or returned by an endpoint.

Replay, record creation, and final decisions remain reviewer-controlled. A production rollout needs explicit retention, deletion, backup, regional-residency, and audit-export policies before real invoice data is accepted.

## 9. Operational Readiness

| Signal                   | SLO or alert                                                              | Owner         | Launch gate |
|--------------------------|---------------------------------------------------------------------------|---------------|-------------|
| Job completion rate      | Track complete, failed, and retry outcomes by extractor backend.          | Project owner | Required    |
| Processing latency       | Capture duration and report p95 by document type and method.              | Project owner | Required    |
| Extraction quality       | Run vendor-separated field accuracy and safe-abstention evaluation.       | Project owner | Required    |
| Fallback rate            | Alert when local-model failures cause unexpected Groq fallback.           | Project owner | Recommended |
| Reconciliation integrity | Run synthetic end-to-end duplicate, quantity, price, and override checks. | Project owner | Required    |

## 10. Alternatives Considered

| Alternative               | Why considered                         | Why not selected                                                                             |
|---------------------------|----------------------------------------|----------------------------------------------------------------------------------------------|
| PDF question answering    | Fast way to retrieve invoice text.     | Does not model line items, evidence, reconciliation checks, or reviewer decisions.           |
| LLM-only extraction       | Generalizes across layouts and labels. | Requires local schema and source-evidence validation; cannot be the sole approval mechanism. |
| Rules-only extraction     | Transparent and inexpensive baseline.  | Breaks on unfamiliar labels and layouts, so it is compared with model-assisted extractors.   |
| Manual spreadsheet review | Simple and familiar.                   | Does not provide durable source links, repeatable checks, or case-level audit history.       |

## 11. Open Questions

What authentication and reviewer roles are required before any non-synthetic invoice is accepted?

Which ERP, receiving, and contract systems should become authoritative upstream sources?

What quality, latency, and cost thresholds justify routing traffic to the fine-tuned adapter?

What retention, deletion, residency, and audit-export policy applies to uploaded supplier documents?

## 12. Decision and Next Steps

Adopt the invoice-first, evidence-backed workflow as the project baseline. First, stabilize document and record linking with synthetic data; second, expand the benchmark with reviewed vendor-separated examples; third, compare rules, Groq, and the QLoRA adapter under identical validation; and finally, add migrations, access control, object storage, and deployment monitoring before considering a limited real-data pilot.

## 13. README for the Repository

LedgerLens is an AI-assisted invoice reconciliation service for procure-to-pay operations. Upload an invoice first; the system extracts fields with page evidence, checks saved supplier and purchasing records, requests only missing supporting documents, and creates a review case when the bundle is ready.

## Core capabilities

- Asynchronous PDF and image intake with SHA-256 de-duplication.
- Embedded PDF text extraction with optional Tesseract OCR for scans.
- Rules-based structured extraction with optional Groq comparison and a QLoRA local adapter.
- Deterministic checks for supplier, currency, PO lines, quantities, accepted receipts, price tolerance, and arithmetic.
- Evidence viewer and human approval or rejection with audit events.

## Run locally

Create a Python environment, install the project extras, start the FastAPI service, and run the worker in a second terminal. Open the local review page or /docs. For the production-shaped stack, run Docker Compose with PostgreSQL, the API, and the durable worker.

## Evaluation

Run the labeled benchmark to report field-level accuracy and failures. Keep validation vendor-separated, include malformed documents to measure safe abstention, and compare rules, Groq, and the local adapter using the same evidence validation.

## Safety boundary

Extraction and matching produce suggestions. A reviewer must confirm a purchasing record and make the final case decision. Do not expose the local app publicly or commit provider keys.

# 14. Interview Questions and Answer Points

| Question                                                   | Answer points                                                                                                                                                               |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why is this more than PDF question answering?              | It extracts typed fields with evidence, matches multiple business records, runs deterministic financial checks, and records a human decision trail.                         |
| How do you prevent an LLM from inventing invoice values?   | Require quoted evidence on a cited page, validate output with Pydantic, reject unsupported values, and keep deterministic rules as the stored baseline.                     |
| How does the invoice-first flow work?                      | The UI uploads the invoice, polls the durable job, asks the requirements endpoint which record is missing, and requests only that next document.                            |
| How do you handle duplicate invoices?                      | Content hashes prevent duplicate document storage; invoice number plus matched supplier is checked before confirmation, and the reviewer sees the duplicate warning.        |
| Why use a worker instead of processing in the API request? | OCR and model inference can be slow or fail. A queued job gives retries, status, bounded failures, and safe recovery independent of the browser request.                    |
| What does needs_review mean?                               | At least one reconciliation check is an exception or a required link is unresolved. Approval is blocked unless a reviewer supplies an explicit override reason.             |
| How would you productionize it?                            | Add authentication and roles, tenant isolation, migrations, managed object storage, secret management, retention rules, monitoring, backups, and a real ERP integration.    |
| How would you justify fine-tuning?                         | Compare prompt, RAG, and QLoRA on a held-out vendor-separated set; use quality, abstention, latency, and cost to decide whether the adapter should replace a provider call. |